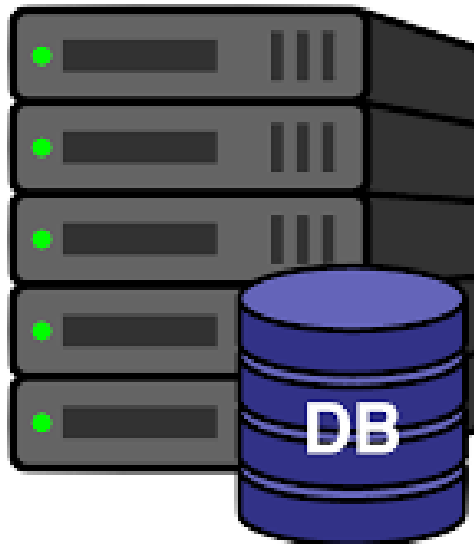




IL-Program. BD-TP-4



G2-4

Réalisé par : Driss EL BARAKA

Encadré par : Pr. Zouiten

Compte Rendu TP4 : Gestion des Augmentations et Projets d'Entreprise

Partie 1 : Création des tables

Objectif : Créer la structure de base pour le TP.

Code :

```
CREATE TABLE DEPARTEMENT (  
    DeptID NUMBER PRIMARY KEY,  
    NomDept VARCHAR2(50),  
    Budget NUMBER  
);  
  
CREATE TABLE EMPLOYE (  
    EmpID NUMBER PRIMARY KEY,  
    Nom VARCHAR2(50),  
    Salaire NUMBER,  
    DeptID NUMBER,  
    CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES DEPARTEMENT(DeptID)  
);  
  
CREATE TABLE ARCHIVE_AUGMENTATION (  
    EmpID NUMBER,  
    AncienSalaire NUMBER,  
    NouveauSalaire NUMBER,  
    DateModification DATE DEFAULT SYSDATE,  
    Utilisateur VARCHAR2(30) DEFAULT USER  
);  
  
INSERT ALL  
    INTO DEPARTEMENT (DeptID, NomDept, Budget) VALUES (10, 'Informatique', 100000)  
    INTO DEPARTEMENT (DeptID, NomDept, Budget) VALUES (20, 'Commercial', 80000)  
    INTO DEPARTEMENT (DeptID, NomDept, Budget) VALUES (30, 'Comptabilite', 60000)  
SELECT * FROM DUAL;  
  
INSERT ALL  
    INTO EMPLOYE (EmpID, Nom, Salaire, DeptID) VALUES (1, 'Alice', 4500, 10)  
    INTO EMPLOYE (EmpID, Nom, Salaire, DeptID) VALUES (2, 'Bob', 3500, 10)  
    INTO EMPLOYE (EmpID, Nom, Salaire, DeptID) VALUES (3, 'Charlie', 3000, 20)  
    INTO EMPLOYE (EmpID, Nom, Salaire, DeptID) VALUES (4, 'Diana', 3200, 30)  
SELECT * FROM DUAL;  
  
COMMIT;
```

Résultat d'exécution :

```
C:\Users\Sky\Desktop\EST\S4\BD\TP4>echo > tp4_partie1.sql

C:\Users\Sky\Desktop\EST\S4\BD\TP4>sqlplus

SQL*Plus: Release 11.2.0.2.0 Production on Mer. Mars 11 13:23:56 2026

Copyright (c) 1982, 2014, Oracle. All rights reserved.

Enter user-name: hr/hr

Connected to:
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production

SQL> @C:\Users\Sky\Desktop\EST\S4\BD\TP4\tp4_partie1.sql

Table created.

Table created.

Table created.

3 rows created.

4 rows created.

Commit complete.
```

Partie 2 : Les Curseurs

Objectif : Calculer et afficher la somme des salaires pour chaque département.

Code :

```

SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_dept IS
        SELECT DeptID, NomDept FROM DEPARTEMENT;

    v_dept_id DEPARTEMENT.DeptID%TYPE;
    v_nom_dept DEPARTEMENT.NomDept%TYPE;
    v_nb_employes NUMBER;
    v_total_salaire NUMBER;
BEGIN
    OPEN c_dept;

    LOOP
        FETCH c_dept INTO v_dept_id, v_nom_dept;
        EXIT WHEN c_dept%NOTFOUND;

        SELECT COUNT(*), NVL(SUM(Salaire), 0)
        INTO v_nb_employes, v_total_salaire
        FROM EMPLOYE
        WHERE DeptID = v_dept_id;

        DBMS_OUTPUT.PUT_LINE('Département: ' || v_nom_dept);
        DBMS_OUTPUT.PUT_LINE('    Nombre d''employés: ' || v_nb_employes);
        DBMS_OUTPUT.PUT_LINE('    Masse salariale: ' || v_total_salaire);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

    CLOSE c_dept;
END;
/

```

Résultat d'exécution :

```

SQL> @C:\Users\Sky\Desktop\EST\S4\BD\TP4\tp4_partie2.sql
Département: Informatique
Nombre d'employés: 2
Masse salariale: 8000
-----
Département: Commercial
Nombre d'employés: 1
Masse salariale: 3000
-----
Département: Comptabilite
Nombre d'employés: 1
Masse salariale: 3200
-----
PL/SQL procedure successfully completed.

```

Analyse :

- Curseur explicite pour parcourir les départements
- NVL(SUM(Salaire), 0) garantit un affichage de 0 pour les départements sans employé
- Rapport structuré avec DBMS_OUTPUT

Partie 3 : Les Procédures et Modes de Paramètres

Objectif : Créer une procédure d'augmentation générique avec paramètres IN, OUT et IN OUT.

Code de la procédure :

```
CREATE OR REPLACE PROCEDURE AugmenterSalaire (  
    p_EmpID IN EMPLOYE.EmpID%TYPE,  
    p_Pourcentage IN NUMBER,  
    p_Résultat OUT NUMBER,  
    p_Message IN OUT VARCHAR2  
) IS  
    v_ancien_salaire EMPLOYE.Salaire%TYPE;  
    v_nouveau_salaire EMPLOYE.Salaire%TYPE;  
    v_nom_employe EMPLOYE.Nom%TYPE;  
    v_prefixe VARCHAR2(50);  
BEGIN  
    -- Sauvegarder le préfixe fourni en entrée  
    v_prefixe := p_Message;  
  
    -- Vérifier si l'employé existe  
    BEGIN  
        SELECT Nom, Salaire INTO v_nom_employe, v_ancien_salaire  
        FROM EMPLOYE  
        WHERE EmpID = p_EmpID;  
    EXCEPTION  
        WHEN NO_DATA_FOUND THEN  
        p_Résultat := -1;  
        p_Message := v_prefixe || ' - ERREUR : Employé ' || p_EmpID || ' inexistant';  
        RETURN;  
    END;  
  
    -- Calculer le nouveau salaire  
    v_nouveau_salaire := v_ancien_salaire * (1 + p_Pourcentage/100);  
  
    -- Mettre à jour la table EMPLOYE  
    UPDATE EMPLOYE  
    SET Salaire = v_nouveau_salaire  
    WHERE EmpID = p_EmpID;
```

```

-- Insérer une trace dans ARCHIVE_AUGMENTATION
INSERT INTO ARCHIVE_AUGMENTATION (EmpID, AncienSalaire, NouveauSalaire)
VALUES (p_EmpID, v_ancien_salaire, v_nouveau_salaire);

-- Succès
p_Résultat := 0;
p_Message := v_prefixe || ' - SUCCÈS : ' || v_nom_employe ||
              '(ID:' || p_EmpID || ') augmenté de ' || p_Pourcentage ||
              '% : ' || v_ancien_salaire || ' € ? ' || v_nouveau_salaire || ' €';

COMMIT;
EXCEPTION
  WHEN OTHERS THEN
    p_Résultat := -2;
    p_Message := v_prefixe || ' - ERREUR TECHNIQUE : ' || SQLERRM;
    ROLLBACK;
END AugmenterSalaire;
/

```

Code de test (bloc anonyme) :

```

SET SERVEROUTPUT ON;

DECLARE
  v_code NUMBER;
  v_message VARCHAR2(200);
BEGIN
  -- Test 1 : Augmentation d'Alice (employé 1) de 10%
  v_message := '[TEST 1]';
  DBMS_OUTPUT.PUT_LINE('=== Test 1 : Augmentation Alice 10% ===');
  AugmenterSalaire(1, 10, v_code, v_message);
  DBMS_OUTPUT.PUT_LINE('Code retour : ' || v_code);
  DBMS_OUTPUT.PUT_LINE('Message : ' || v_message);
  DBMS_OUTPUT.PUT_LINE('');

  -- Test 2 : Employé inexistant (99)
  v_message := '[TEST 2]';
  DBMS_OUTPUT.PUT_LINE('=== Test 2 : Employé inexistant 99 ===');
  AugmenterSalaire(99, 5, v_code, v_message);
  DBMS_OUTPUT.PUT_LINE('Code retour : ' || v_code);
  DBMS_OUTPUT.PUT_LINE('Message : ' || v_message);
  DBMS_OUTPUT.PUT_LINE('');

  -- Vérification du contenu de ARCHIVE_AUGMENTATION
  DBMS_OUTPUT.PUT_LINE('=== Contenu de ARCHIVE_AUGMENTATION ===');
  FOR rec IN (SELECT * FROM ARCHIVE_AUGMENTATION ORDER BY DateModification) LOOP
    DBMS_OUTPUT.PUT_LINE('EmpID: ' || rec.EmpID ||
                          ', Ancien: ' || rec.AncienSalaire ||
                          ', Nouveau: ' || rec.NouveauSalaire ||
                          ', Date: ' || TO_CHAR(rec.DateModification, 'DD/MM/YYYY HH24:MI:SS') ||
                          ', User: ' || rec.Utilisateur);
  END LOOP;

  -- Afficher les employés après modification
  DBMS_OUTPUT.PUT_LINE('');
  DBMS_OUTPUT.PUT_LINE('=== Employés après augmentation ===');
  FOR rec IN (SELECT * FROM EMPLOYE ORDER BY EmpID) LOOP
    DBMS_OUTPUT.PUT_LINE(rec.EmpID || ' - ' || rec.Nom || ' - ' || rec.Salaire || ' € - Dept ' || rec.DeptID);
  END LOOP;
END;

```

Résultat d'exécution :

```

SQL> @C:\Users\Sky\Desktop\EST\S4\BD\TP4\tp4_partie3_anonyme.sql
=== Test 1 : Augmentation Alice 10% ===
Code retour : 0
Message : [TEST 1] - SUCCES : Alice (ID:1) augmenté de 10% : 5445 Ç ? 5989,5 Ç
=== Test 2 : Employé inexistant 99 ===
Code retour : -1
Message : [TEST 2] - ERREUR : Employé 99 inexistant
=== Contenu de ARCHIVE_AUGMENTATION ===
EmpID: 1, Ancien: 4500, Nouveau: 4950, Date: 11/03/2026 14:13:04, User: HR
EmpID: 1, Ancien: 4950, Nouveau: 5445, Date: 11/03/2026 14:14:23, User: HR
EmpID: 1, Ancien: 5445, Nouveau: 5989,5, Date: 11/03/2026 14:15:32, User: HR
=== Employés après augmentation ===
1 - Alice - 5989,5 Ç - Dept 10
2 - Bob - 3500 Ç - Dept 10
3 - Charlie - 3000 Ç - Dept 20
4 - Diana - 3200 Ç - Dept 30

PL/SQL procedure successfully completed.

```

Analyse :

- **Paramètre IN** : p_EmpID, p_Pourcentage (valeurs fournies à la procédure)
- **Paramètre OUT** : p_Resultat (code retourné par la procédure)
- **Paramètre IN OUT** : p_Message (préfixe en entrée, message complet en sortie)
- Gestion des exceptions : NO_DATA_FOUND pour employé inexistant, OTHERS pour erreurs techniques
- Archivage automatique des augmentations

Partie 4 : Les Fonctions

Objectif : Créer une fonction pour vérifier la cohérence du budget.

Code de la fonction :

tp4_partie4_function.sql - Bloc-notes

Fichier Edition Format Affichage Aide

```
CREATE OR REPLACE FUNCTION VerifierBudgetDept (
    p_DeptID IN DEPARTEMENT.DeptID%TYPE
) RETURN BOOLEAN
IS
    v_total_salaires NUMBER;
    v_budget DEPARTEMENT.Budget%TYPE;
BEGIN
    -- Récupérer le budget du département
    BEGIN
        SELECT Budget INTO v_budget
        FROM DEPARTEMENT
        WHERE DeptID = p_DeptID;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN FALSE;
    END;

    -- Calculer la somme des salaires du département
    SELECT NVL(SUM(Salaire), 0) INTO v_total_salaires
    FROM EMPLOYE
    WHERE DeptID = p_DeptID;

    -- Comparer et retourner le résultat
    IF v_total_salaires <= v_budget THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END VerifierBudgetDept;
/
```

Code de test (bloc anonyme) :

Fichier Edition Format Affichage Aide

```
SET SERVEROUTPUT ON;

DECLARE
    FUNCTION test_budget(p_dept NUMBER) RETURN VARCHAR2 IS
    BEGIN
        IF VerifierBudgetDept(p_dept) THEN
            RETURN 'Budget OK';
        ELSE
            RETURN 'Budget dépasse ou departement inexistant';
        END IF;
    END;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Departement 10 : ' || test_budget(10));
    DBMS_OUTPUT.PUT_LINE('Departement 30 : ' || test_budget(30));
    DBMS_OUTPUT.PUT_LINE('Departement 99 : ' || test_budget(99));
END;
/
```

Résultat d'exécution :


```
SQL> @C:\Users\Sky\Desktop\EST\S4\BD\TP4\tp4_partie4_anonyme.sql
Departement 10 : Budget OK
Departement 30 : Budget OK
Departement 99 : Budget depasse ou departement inexistant

PL/SQL procedure successfully completed.
```

Analyse :

- Fonction retournant un booléen (TRUE/FALSE)
- Vérification de l'existence du département
- Calcul de la somme des salaires avec NVL pour éviter NULL
- Comparaison avec le budget disponible

Partie 5 : Les Triggers (à compléter)

Objectif : Automatiser l'archivage des augmentations.

Code du trigger :

```
CREATE OR REPLACE TRIGGER TRG_ARCHIVE_SALAIRE
  AFTER UPDATE OF Salaire ON EMPLOYEE
  FOR EACH ROW
BEGIN
  INSERT INTO ARCHIVE_AUGMENTATION (EmpID, AncienSalaire, NouveauSalaire)
  VALUES (:OLD.EmpID, :OLD.Salaire, :NEW.Salaire);
END;
/
```

Modification de la procédure : (supprime d l'INSERT dans AugmenterSalaire)

```

CREATE OR REPLACE PROCEDURE AugmenterSalaire (
    p_EmpID IN EMPLOYEE.EmpID%TYPE,
    p_Pourcentage IN NUMBER,
    p_Résultat OUT NUMBER,
    p_Message IN OUT VARCHAR2
) IS
    v_ancien_salaire EMPLOYEE.Salaire%TYPE;
    v_nouveau_salaire EMPLOYEE.Salaire%TYPE;
    v_nom_employe EMPLOYEE.Nom%TYPE;
    v_prefixe VARCHAR2(50);
BEGIN
    v_prefixe := p_Message;

    -- Vérifier si l'employé existe
    BEGIN
        SELECT Nom, Salaire INTO v_nom_employe, v_ancien_salaire
        FROM EMPLOYEE
        WHERE EmpID = p_EmpID;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            p_Résultat := -1;
            p_Message := v_prefixe || ' - ERREUR : Employé ' || p_EmpID || ' inexistant';
            RETURN;
    END;

    -- Calculer le nouveau salaire
    v_nouveau_salaire := v_ancien_salaire * (1 + p_Pourcentage/100);

    -- Mettre à jour la table EMPLOYEE (le trigger se charge de l'archivage)
    UPDATE EMPLOYEE
    SET Salaire = v_nouveau_salaire
    WHERE EmpID = p_EmpID;

    -- Succès
    p_Résultat := 0;
    p_Message := v_prefixe || ' - SUCCÈS : ' || v_nom_employe ||
        ' (ID: ' || p_EmpID || ') augmenté de ' || p_Pourcentage ||
        '% : ' || v_ancien_salaire || ' € ? ' || v_nouveau_salaire || ' €';

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        p_Résultat := -2;
        p_Message := v_prefixe || ' - ERREUR TECHNIQUE : ' || SQLERRM;
        ROLLBACK;
END AugmenterSalaire;
/

```

Test : Réexécuter la procédure et vérifier que l'archive se remplit via le trigger.

```

SQL> @C:\Users\Sky\Desktop\EST\S4\BD\TP4\tp4_partie3.sql

Procedure created.

SQL>

```

```

SQL> @C:\Users\Sky\Desktop\EST\S4\BD\TP4\tp4_partie3_anonyme.sql
=== Test 1 : Augmentation Alice 10% ===
Code retour : 0
Message : [TEST 1] - SUCCÈS : Alice (ID: 1) augmenté de 10% : 6588,45 Ç ?
7247,295 Ç
=== Test 2 : Employé inexistant 99 ===
Code retour : -1
Message : [TEST 2] - ERREUR : Employé 99 inexistant
=== Contenu de ARCHIVE_AUGMENTATION ===
EmpID: 1, Ancien: 4500, Nouveau: 4950, Date: 11/03/2026 14:13:04, User: HR
EmpID: 1, Ancien: 4950, Nouveau: 5445, Date: 11/03/2026 14:14:23, User: HR
EmpID: 1, Ancien: 5445, Nouveau: 5989,5, Date: 11/03/2026 14:15:32, User: HR
EmpID: 1, Ancien: 6588,45, Nouveau: 7247,295, Date: 18/03/2026 11:38:20, User:
HR
=== Employés après augmentation ===
1 - Alice - 7247,295 Ç - Dept 10
2 - Bob - 3500 Ç - Dept 10
3 - Charlie - 3000 Ç - Dept 20
4 - Diana - 3200 Ç - Dept 30

PL/SQL procedure successfully completed.

SQL>

```

Conclusion

Ce TP a permis de manipuler :

- **Curseurs explicites** pour parcourir les départements
- **Procédures** avec paramètres IN, OUT et IN OUT
- **Fonctions** retournant des booléens
- **Gestion d'exceptions** (NO_DATA_FOUND, OTHERS)
- **Transactions** (COMMIT, ROLLBACK)
- **Archivage** des modifications